Text Mining

4DT910/4DV510: Data Mining

Outline

- What is Text Mining?
- What Can it Do?
- Text Mining Pipeline
- Mining Approaches
- Two Real Examples

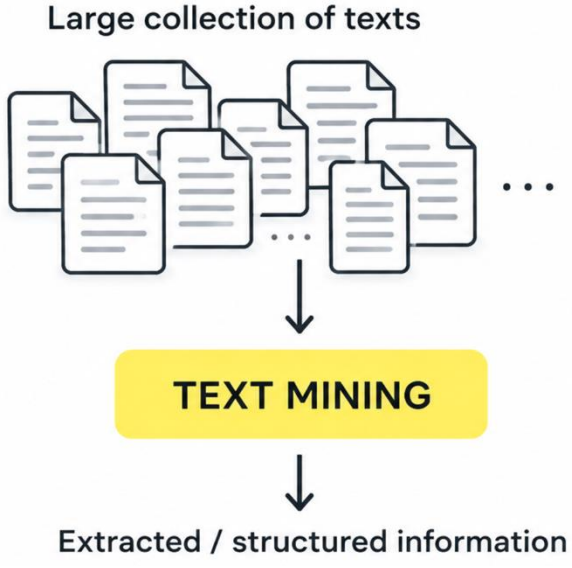
What is Text Mining?

Text Mining is the process of transforming unstructured text into more structured information by extracting and organizing the information needed for analysis.

It combines techniques from Natural Language Processing (NLP), Machine Learning and Statistics to process large collections of text automatically.

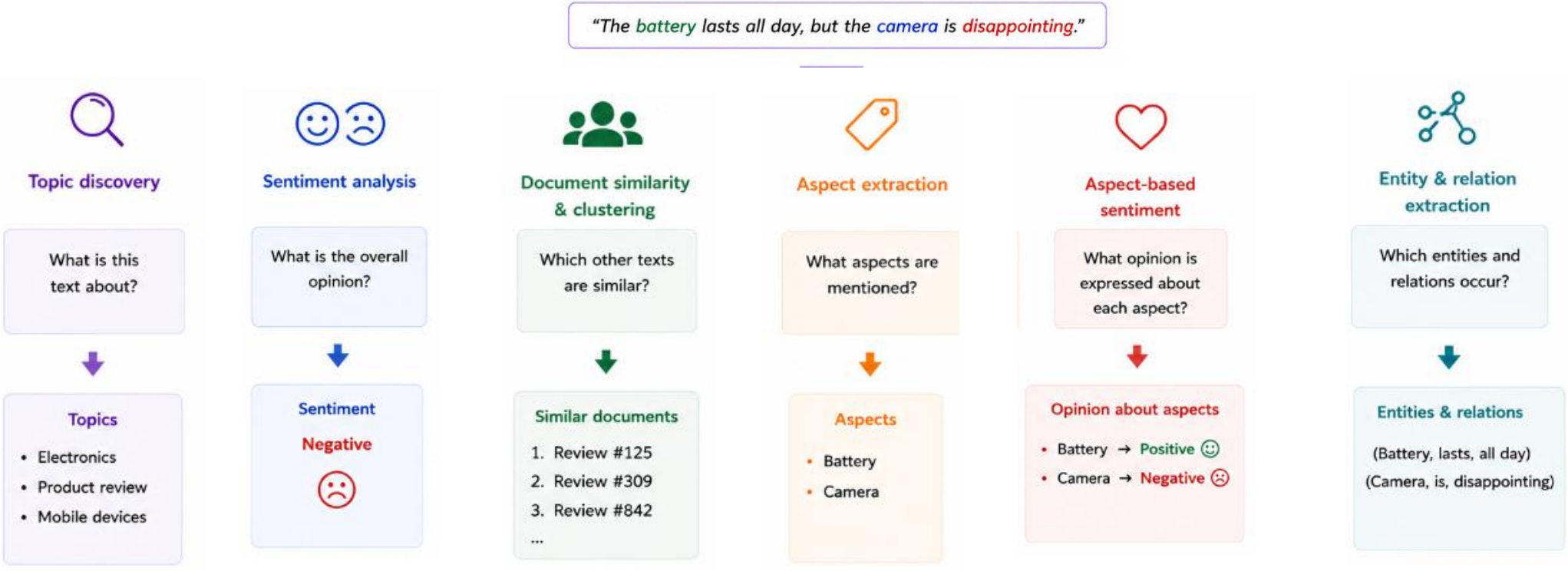
Why is it important?

- Most real-world data is unstructured text (clinical notes, emails, reports, social media, etc.).
- Manual reading of large text collections is time-consuming and inconsistent.
- Text mining makes it possible to extract relevant information faster and in a structured form, supporting analysis and decision making.

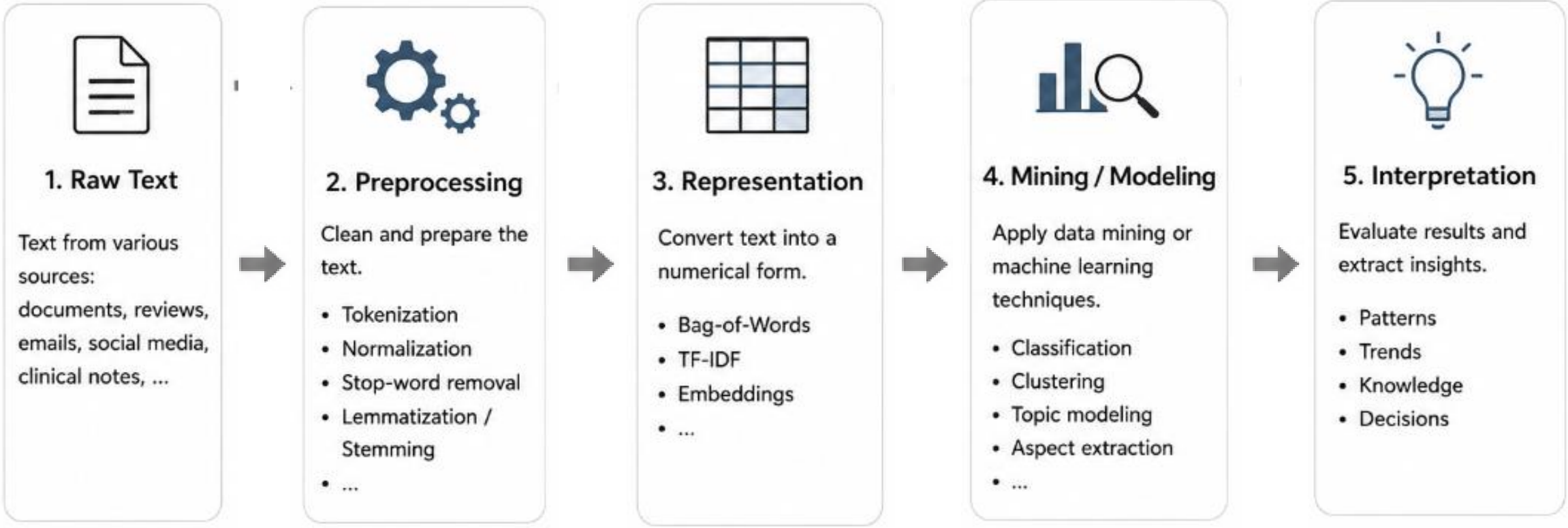


What Can We Discover from Text?

From the **same text**, we can ask **different questions** and discover different kinds of information



Text Mining Pipeline



Text Mining Approaches Overview



Text Preprocessing (Optional)

Cleaning



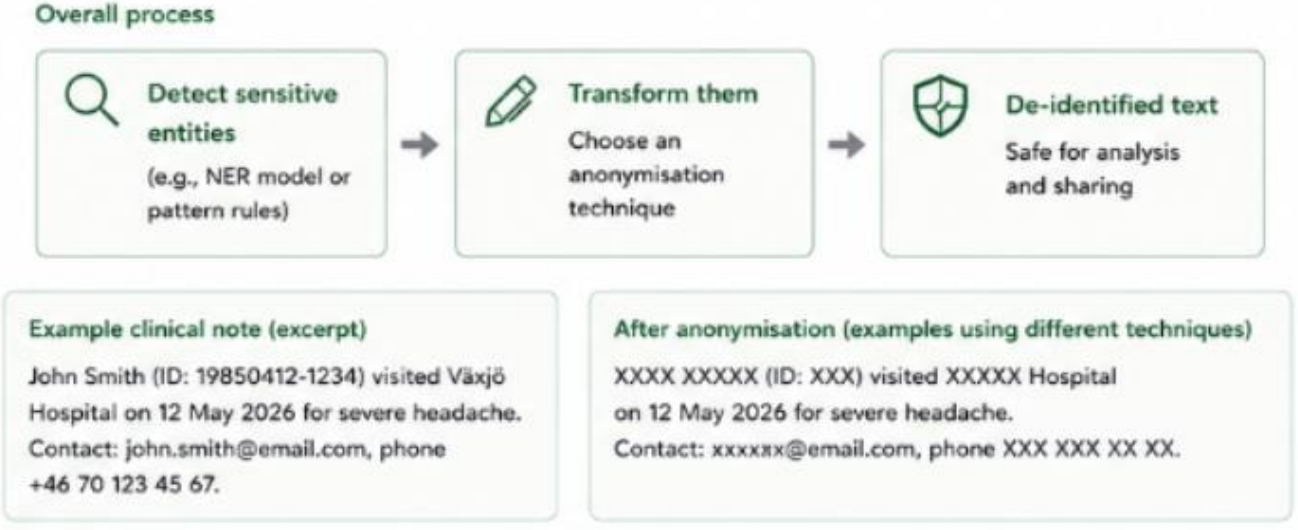
Typical cleaning steps

- Lowercase
- Remove or replace URLs
- Remove or replace @mentions
- Process hashtags (#NewPhone → newphone)
- Remove extra punctuation and repeated characters (soooo → so)
- Handle emojis (keep or map to tokens if useful)
- Remove HTML / special characters

Improves data quality

Anonymization

Protect privacy removing or masking personal and institutional identifiers before analysis or sharing for analysis



https://hbdeid.dsv.su.se/

Text Preprocessing

Example text:
"The patients received new medications, but the treatment did not help."

1 Tokenization

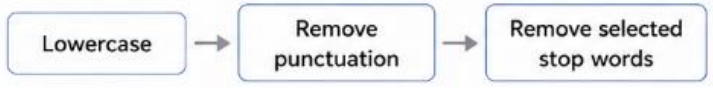
Split text into smaller units called tokens.




```
["The", "patients", "received", "new",
"medications", ",", "but", "the", "treatment",
"did", "not", "help", "."]
```

2 Normalization / Cleaning

Standardize the text and optionally remove elements that are not useful for the task.




```
["patients", "received", "new", "medications",
"treatment", "not", "help"]
```

Stop-word removal is task-dependent.
Removing **not** could change the meaning.

Language matters

English

- relatively limited word inflection
- many established stop-word lists

Swedish

- more inflected forms
  bil → bilen → bilar → bilarna
- compound words are common
  sjuk + sköterska → sjuksköterska
- preserve å, ä, ö
- requires Swedish stop-word lists and language-specific tools

Preprocessing depends on the language, task and model.

Text Preprocessing

3 Normalization using stemming or lemmatization

Reduce words to their base or root form.

Stemming (rule-based)

Chops word endings to obtain word stems.

```
["patient", "receiv", "new", "medic",
"treatment", "not", "help"]
```

Original	Stemmed
patients	patient
received	receiv
medications	medic

ⓘ Fast and simple, but may produce non-words.

Lemmatization (dictionary & morphology-based)

Uses vocabulary and morphological analysis to get base forms.

```
["patient", "receive", "new", "medication",
"treatment", "not", "help"]
```

Original	Lemmatized
patients	patient
received	receive
medications	medication

ⓘ More accurate and meaningful base forms.

Text Preprocessing for Pretrained Encoder Models

1. Clean documents



- Remove irrelevant content (e.g., HTML/XML tags, duplicated headers)
- Fix encoding issues
- Normalize whitespace and line breaks
- Remove corrupted characters

2. Segment text



- Keep the original text
- Split into sentences, paragraphs, or smaller chunks if the document is long
- Ensure each chunk contains a coherent piece of text

3. Model tokenizer



- Use the tokenizer that belongs to the pretrained model
- Convert text to subword tokens (e.g., WordPiece, BPE)
- Tokenizer adds special tokens (e.g., [CLS], [SEP])
- Do not manually split words

4. Handle length



- Truncate sequences that exceed the maximum length (e.g., 512 tokens)
- Pad shorter sequences when processing in batches
- Create attention masks (to indicate real tokens vs. padding)

Usually NOT needed (unlike classical text mining)

- Stop-word removal
- Stemming or lemmatization
- Punctuation removal
- Lowercasing (depends on the model)
- TF-IDF or n-gram features
- Spelling correction (usually not automatic)

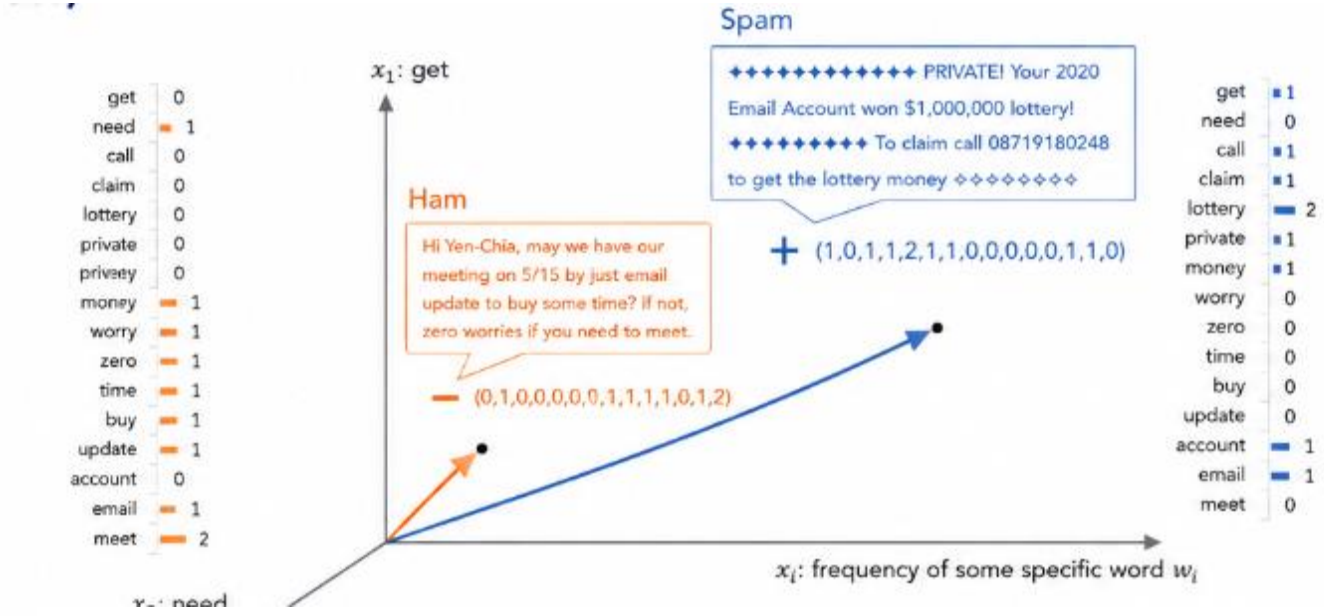
Representation: Bag of Words

Now we have cleaned tokens that represent a sentence. We need to transform them to data points in some high-dimensional space.

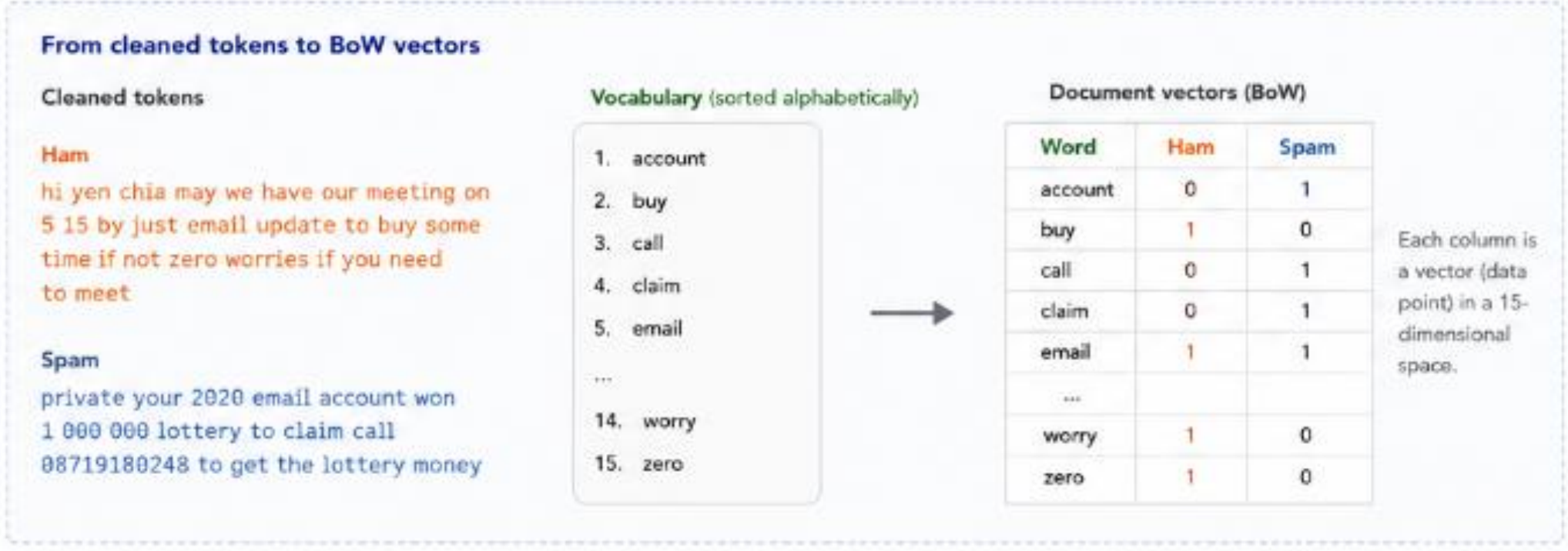
These data points are also called **vectors**, which means arrays of numbers that encode both the direction and length information.

Bag of Words idea

Create a vocabulary (all unique words in the corpus). Represent each document as a vector of word frequencies (counts).



Representation: Bag of Words



Representation: TF-IDF

TF-IDF

TF-IDF (Term Frequency – Inverse Document Frequency) weights terms by how important they are in a document relative to a whole collection of documents.

$$w_{t,d} = \underbrace{\mathrm{tf}(t,d)}_{\text{The more frequently a term appears in a given document...}} \times \underbrace{\mathrm{idf}(t,D)}_{\text{...and the fewer times it appears in other documents...}}$$

The higher its TF-IDF value.

tf(t, d) — **Term Frequency:** how often term t occurs in document d.

Common choice:

$$tf(t,d) = \frac{f_{t,d}}{\sum_{k \in d} f_{k,d}}$$

- $f_{t,d}$ – raw count of term t in d
- $\sum_{k \in d} f_{k,d}$ – total number of terms in document d

This normalizes by document length so that longer documents do not get unfairly higher scores.

idf(t, D) — **Inverse Document Frequency:** how rare term t is across the collection of D documents.

Common choice:

$$idf(t,D) = \log\left(\frac{N}{df_t}\right)$$

- N – total number of documents
- df_t – number of documents containing term t

Smoothing is often used to avoid division by zero, e.g., $idf(t,D) = \log\left(\frac{N+1}{df_t+1}\right) + 1$.

Exercise: TF-IDF

Document ID	Term frequency ($tf_{t,d}$)			
	apple	bike	spaceship	tea
doc1	8	0	4	8
doc2	3	2	3	0
doc3	2	3	0	1
doc4	4	0	0	0

TF-IDF definition

$$w_{t,d} = \mathrm{tf}(t,d) \times \mathrm{idf}(t,D)$$

- $\mathrm{tf}(t,d)$: how often term t occurs in document d (often normalized)
- $\mathrm{idf}(t,D)$: how rare term t is across the collection of D documents

$$idf(t,D) = \ln\left(\frac{N}{df_t}\right)$$

N – total number of documents
df_t – number of documents containing term t

Here: $N = 4$ (we have four documents: $doc1$–$doc4$)

Step-by-step TF-IDF for *doc1*

1) Term frequency in doc1

$tf(t, \mathrm{doc1})$

Term	tf(t, doc1)
apple	8
bike	0
spaceship	4
tea	8

(Raw counts as given)

2) Document frequency

df_t: in how many documents term t appears at least once

Term	dft
apple	4
bike	2
spaceship	2
tea	2

Count documents with tf > 0

3) Inverse Document Frequency

$$idf(t,D) = \ln\left(\frac{N}{df_t}\right),\ N = 4$$

Term	idf(t, D)
apple	$\ln(4/4) = \ln(1) = 0.000$
bike	$\ln(4/2) = \ln(2) = 0.693$
spaceship	$\ln(4/2) = \ln(2) = 0.693$
tea	$\ln(4/2) = \ln(2) = 0.693$

Natural logarithm

4) TF-IDF score for doc1

$$w_{t,doc1} = \mathrm{tf}(t,\mathrm{doc1}) \times \mathrm{idf}(t,D)$$

Term	TF-IDF (doc1)
apple	$8 \times 0.000 = 0.000$
bike	$0 \times 0.693 = 0.000$
spaceship	$4 \times 0.693 = 2.773$
tea	$8 \times 0.693 = 5.545$

Multiply tf by idf

5) Most representative word for doc1

Compare the TF-IDF scores in the previous column.

Highest score:
tea (5.545)

Representation: N-grams

Example sentence (after cleaning): `the movie was not good`

Unigrams ($n = 1$)
individual words

the, movie, was, not, good

Bigrams ($n = 2$)
pairs of consecutive words

the movie, movie was, was not, not good

Trigrams ($n = 3$)
triples of consecutive words

the movie was, movie was not, was not good

Why use n-grams?

- BoW (unigrams) ignores word order:

  "not good" and "good not" have the same unigrams.

- N-grams keep local order information.
- Larger n captures longer phrases but increases the number of features.

From n-grams to vectors (example with counts)

Vocabulary (bigrams)

```
the movie
movie was
was not
not good
...
```

Document	Bigram counts (feature vector)				
	the movie	movie was	was not	not good	...
doc1	1	1	1	1	...
doc2	0	1	0	1	...
doc3	0	0	1	0	...
...	...	...	...	...	...

Note
We can also compute TF-IDF weights for n-grams exactly as we did for words.



Representations: Word Embeddings

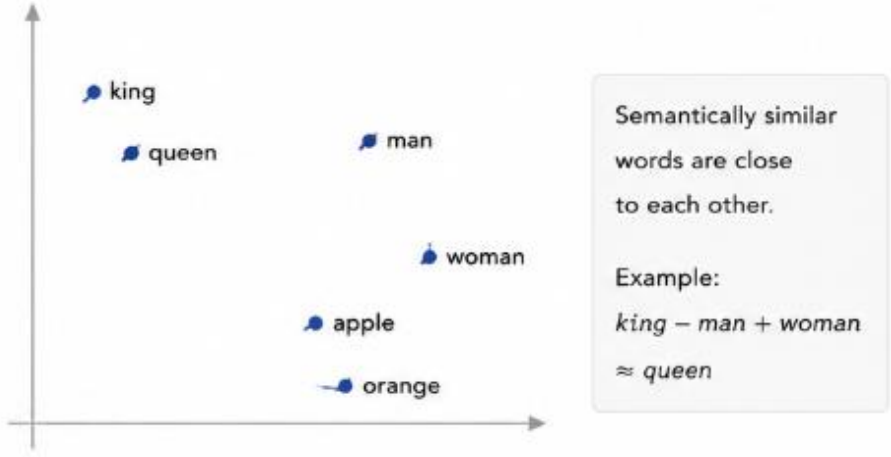
Bag of Words / TF-IDF representation (sparse and high-dimensional)

Vocabulary	apple	banana	computer	...	zebra
apple	1	0	0	...	0
banana	0	1	0	...	0
computer	0	0	1	...	0
...	...	...	...	...	...
zebra	0	0	0	...	1

Each word is a one-hot vector: all zeros except one position.

Similar words are orthogonal (unrelated).

Word embeddings (dense and low-dimensional)



Semantically similar words are close to each other.

Example:

king − man + woman ≈ queen

Example vectors (2-dimensional for illustration)

Word	Vector
king	[0.51, 0.83]
queen	[0.47, 0.79]
man	[0.80, 0.60]
woman	[0.82, 0.55]
apple	[-0.41, -0.72]
orange	[-0.35, -0.67]

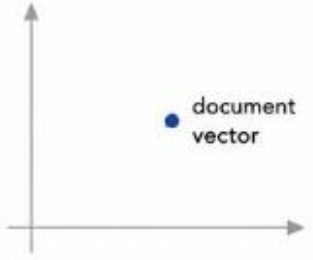
Each row is a dense vector with real values. Dimensions are learned from large text data (e.g., Word2Vec, GloVe, FastText).

Building a document vector

A simple way: average the word vectors in the document.

Document: "*king and queen are royalty*"

≈ average(*king, queen, are, royalty*)



Representations: Word2Vec, GloVe, FastText

Limitations of Word2Vec

1. One vector per word (no context)

Same word → same vector, even in different meanings.

I deposited money at the bank. (financial institution)

We sat on the river bank. (river side)

Word2Vec gives "bank" the same vector in both sentences.

2. Out-of-vocabulary words

If a word was never seen during training, Word2Vec has no vector for it.

Unseen word (e.g., a new drug name) → No Word2Vec vector

Key idea

Word2Vec (and GloVe) produce static word embeddings: one vector per word, regardless of context.

Contextual models (e.g., BERT) can give different representations depending on the sentence.

Word2Vec vs. GloVe vs. FastText

	Word2Vec	GloVe	FastText
Main idea	Learns by predicting words from local context. (context words → target word → context words (CBOW); or target word → context words (Skip-gram))	Learns from global word co-occurrence statistics in the whole corpus. (Word co-occurrence matrix (global) → Dense vectors)	Extends Word2Vec by representing each word as a bag of character n-grams (subwords). Example: *playing* → <pl, pla, lay, ayi, yin, ing, ng> … Word vector = sum/average of its subword vectors
Uses local context	✓	✓	Limited (indirectly)
Uses global co-occurrence		✓	✓
Uses word parts (subwords)			
Handles unseen/ rare words	Poorly		Better (can form vectors for rare/unseen words)
Best at	General tasks, large datasets	Capturing global statistics, analogies	Morphologically rich languages, rare words, misspellings

Example: Swedish morphology

FastText can share information between related forms.

Words:

sjuksköterska
sjuksköterskan
sjuksköterskor
sjuksköterskeutbildning
...

They share many character n-grams, so FastText can learn good vectors even for inflected or compound forms.

Representations: Contextual Embeddings

Static embeddings (Word2Vec, GloVe, FastText)

I deposited money in the **bank**.

We sat on the river **bank**.

bank → □□□□

same vector

Each word has one fixed vector, independent of context.

Contextual embeddings (BERT and other transformer models)

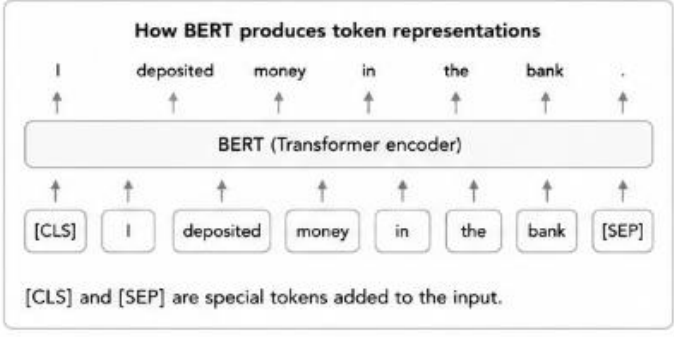
I deposited money in the **bank**. ⟶ [0.21, -0.54, 0.32, …]

We sat on the river **bank**. ⟶ [-0.37, 0.62, -0.11, …]

The representation of a word depends on its context.
Different context → different vector.

What is BERT?

BERT (Bidirectional Encoder Representations from Transformers) is a transformer-based language model.
It reads the entire sentence (both left and right context) to produce contextual representations for each token.



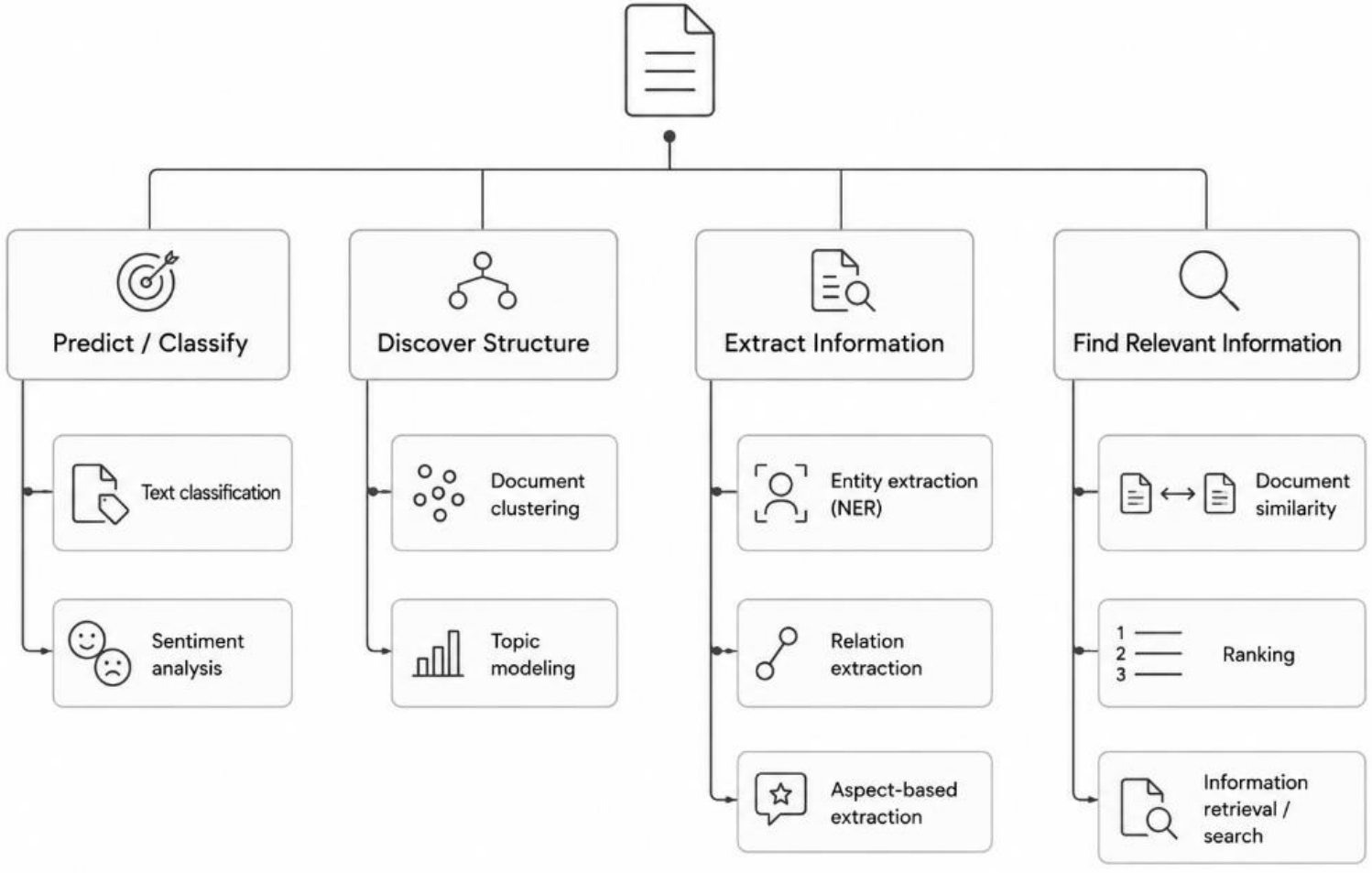
Different levels of representations

□□□□ **Token embeddings**
Representation for each word/piece in its context.

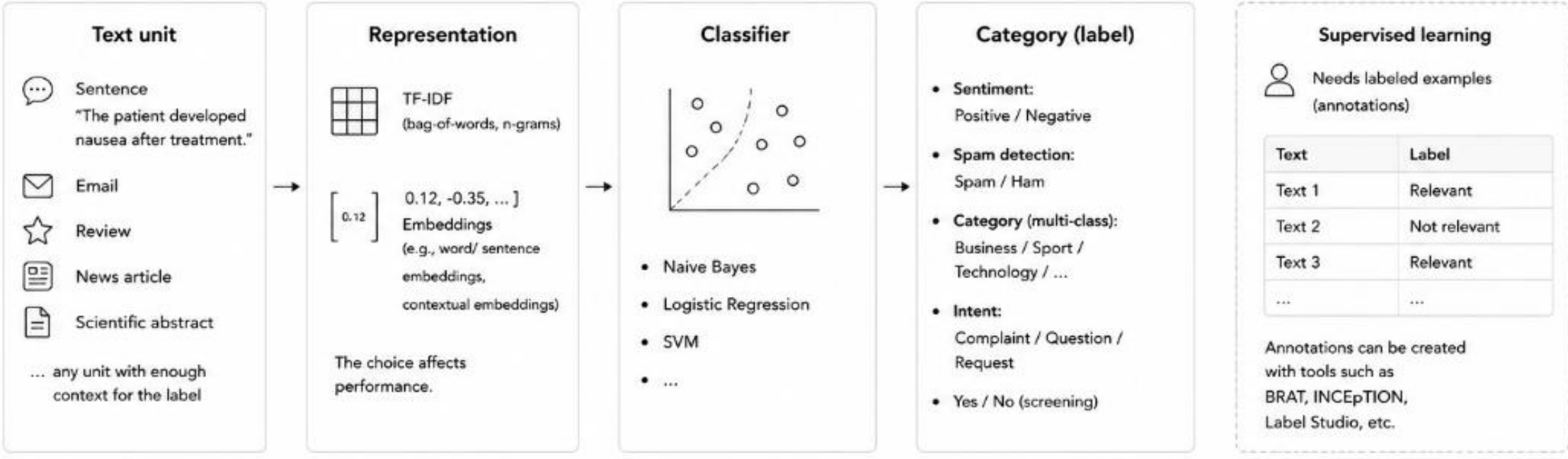
Sentence embeddings
One vector representing the whole sentence (e.g., from [CLS] token or pooling).

Document embeddings
One vector representing the whole document (e.g., pooling over tokens or sentences).

Mining

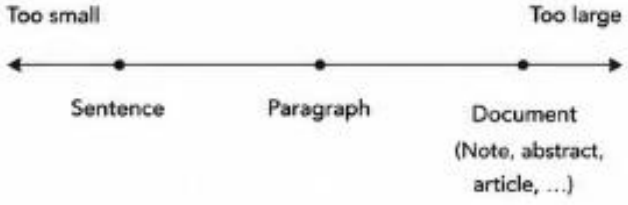

Predict / Classify
Discover Structure
Extract Information
Find Relevant Information
Text classification
Sentiment analysis
Document clustering
Topic modeling
Entity extraction (NER)
Relation extraction
Aspect-based extraction
Document similarity
Ranking
Information retrieval / search

Predict/Classify



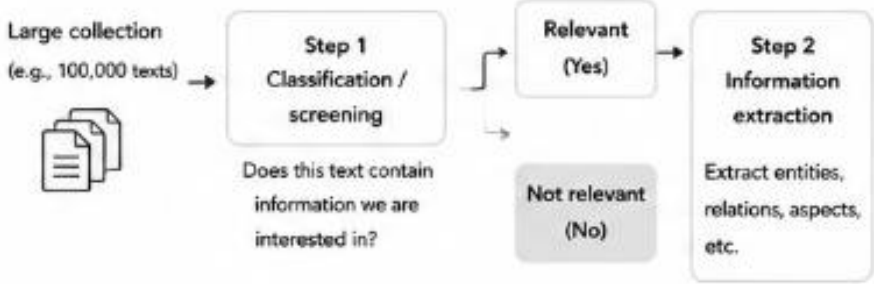
Predict/Classify (Supervised)

How to decide the size of text unit?



- Choose the smallest unit that contains enough context to assign the label reliably.
- **Too small**: may miss important context.
- **Too large**: may mix different topics and make one label unclear.
- It depends on the task and available annotations.

First screening → then information extraction



Example (clinical text)

Text: "Patient developed severe nausea after starting Drug X."

Step 1 (Classification): contains ADE? → Yes

Step 2 (Information extraction): [Drug X] causes → [nausea]

Types of classification

Binary classification

Yes / No, Spam / Ham, Relevant / Not relevant

Multi-class classification

Business / Sport / Technology / Health / ...

Multi-label classification

A text can belong to multiple labels simultaneously (e.g., {AI, Healthcare, Research})

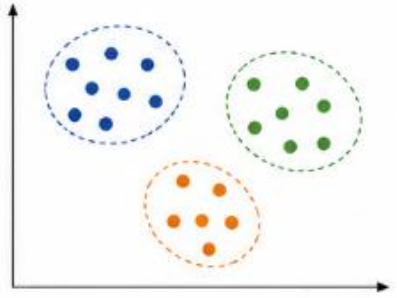
Discover Structure (Unsupervised)

1. Clustering (Unsupervised)

Groups similar texts together based on their semantic similarity, without predefined labels.

How it works

- Represent texts (e.g., TF-IDF, embeddings)
- Compute similarity/distance (e.g., cosine)
- Group texts into clusters (e.g., k-means, hierarchical, DBSCAN)



Applications (examples)

- Discover groups of similar patient comments
- Segment documents by themes or content type
- Detect outliers or unusual cases
- Organize large collections before manual review

Example

Comments about 'waiting time' fall into one cluster, 'staff attitude' into another.

Limitations & issues

- Choosing the right number of clusters (k)
- Sensitive to representation and parameters
- Clusters may mix different themes
- Results can be unstable across runs

Evaluation

No gold standard. Use intrinsic measures (silhouette, cohesion) and manual inspection.

2. Topic Modeling (Unsupervised)

Discovers latent topics that explain the words in the collection. Each document is a mixture of topics.

How it works

Picture

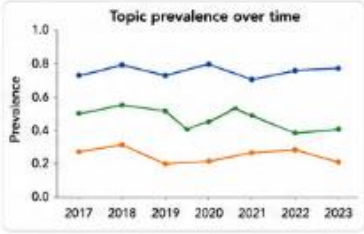
- Model discovers topics as distributions of words (e.g., LDA, NMF)
- Each document gets a distribution over topics
- Topics are summarized by their most probable words

Example topics (top words)

Topic 1 Access	Topic 2 Staff	Topic 3 Follow-up
phone, call, appointment, contact, reach, time, queue, ...	doctor, nurse, staff, friendly, attitude, help, care, treat, ...	follow-up, information, result, test, feedback, reply, continuity, ...

Applications (examples)

- Explore main themes in large text collections
- Track how topics change over time
- Generate data-driven categories/aspect candidates
- Support dashboards and decision making



Limitations & issues

- Number of topics must be chosen
- Topics may be hard to interpret or overlap
- Sensitive to preprocessing and model settings
- Small or imbalanced data reduce quality

Verification & evaluation

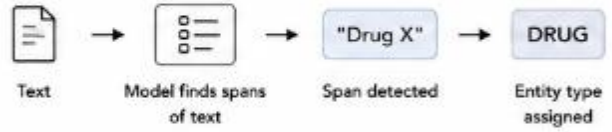
Topics must be interpreted by humans (coherence, diversity). No absolute ground truth—use qualitative assessment and downstream usefulness.

Information Extraction (Supervised)

1. Entities Extraction

Identifies and classifies mentions of entities in text into predefined types.

How it works



Applications (from literature)

- **Clinical:** Named Entity Recognition (NER) for symptoms, diseases, drugs (e.g., cTAKES, MetaMap) [1,2]
- **Biomedical:** Gene/protein/disease recognition in abstracts (e.g., DNorm, SciSpacy) [3]
- **Finance:** Company, organization and location extraction from reports and news [4]
- **Legal:** Case, statute, court and citation extraction [5]

Successful applications

- cTAKES achieved F1-score ~0.85 for clinical NER on i2b2 datasets [1]
- SciSpacy models reach F1 > 0.90 for biomedical NER [3]

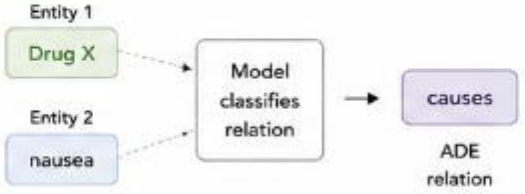
Output example



2. Relation Extraction

Finds and classifies relationships between entities.

How it works



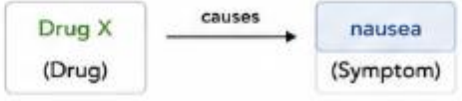
Applications (from literature)

- **Clinical:** Drug–adverse event, drug–indication relations (e.g., ADE extraction from EHRs) [6–8]
- **Biomedical:** Protein–protein interactions, gene–disease relations [9]
- **Social media:** Cause–effect or event–outcome relations (e.g., disaster impact) [10]

Successful applications

- ADE relation extraction systems report F1 = 0.75–0.85 on benchmark corpora (e.g., ADE Corpus) [6,7]
- BioCreative VI relation extraction tasks: F1 up to 0.80 [9]

Output example



3. Aspect-based Extraction (ABSA)

Links aspects of interest in text with the opinion or sentiment expressed about them.

How it works



Applications (from literature)

- **Healthcare:** Patient feedback analysis of services and care (e.g., waiting time, staff attitude) [11]
- **Product reviews:** Extract aspects (battery, camera, price) with sentiments [12]
- **Tourism & hospitality:** Service aspects with sentiment (e.g., cleanliness, location) [13]

Successful applications

- Aspect-based sentiment models achieve F1 = 0.70–0.90 on SemEval restaurant/hotel datasets [12,13]
- Used in real-world dashboards for patient experience monitoring in hospitals and primary care [11]

Output example

Aspect	Sentiment	Polarity score
waiting time	Negative	-0.72
staff	Positive	+0.68

Information Extraction (Supervised)

1. Ambiguity & Context

- Negation, uncertainty and modality change the meaning.
- Relations may span long distances or multiple sentences.
- Same words can have different meanings in different contexts.

Example:

"The patient developed pneumonia."

"No evidence of pneumonia."

"Possible pneumonia."

"The patient started aspirin three weeks ago. During the following days, she developed gastrointestinal bleeding."

2. Annotation

- Creating training data is expensive and time-consuming.
- Annotators may disagree (subjectivity).
- Requires clear guidelines and agreement assessment.

Example:

Text: *"Aspirin caused severe bleeding."*

Annotators must decide:

- What are the entities?
- What are their boundaries?
- What is the relation type?

Different guidelines → different annotations.

3. Model / Data

- Models may not generalize across domains or languages (domain shift).
- Rare entities/relations lead to data sparsity.
- Class imbalance makes learning and detection difficult.

Example:

A model trained on news text may struggle with clinical notes.

10,000 No Relation

vs.

150 ADE Relations

High accuracy can be misleading.

4. Evaluation

- Entity boundary matching is not always straightforward.
- Errors in entity extraction propagate to relation and aspect extraction.
- Choice of metrics strongly affects conclusions.

Example (NER boundary):

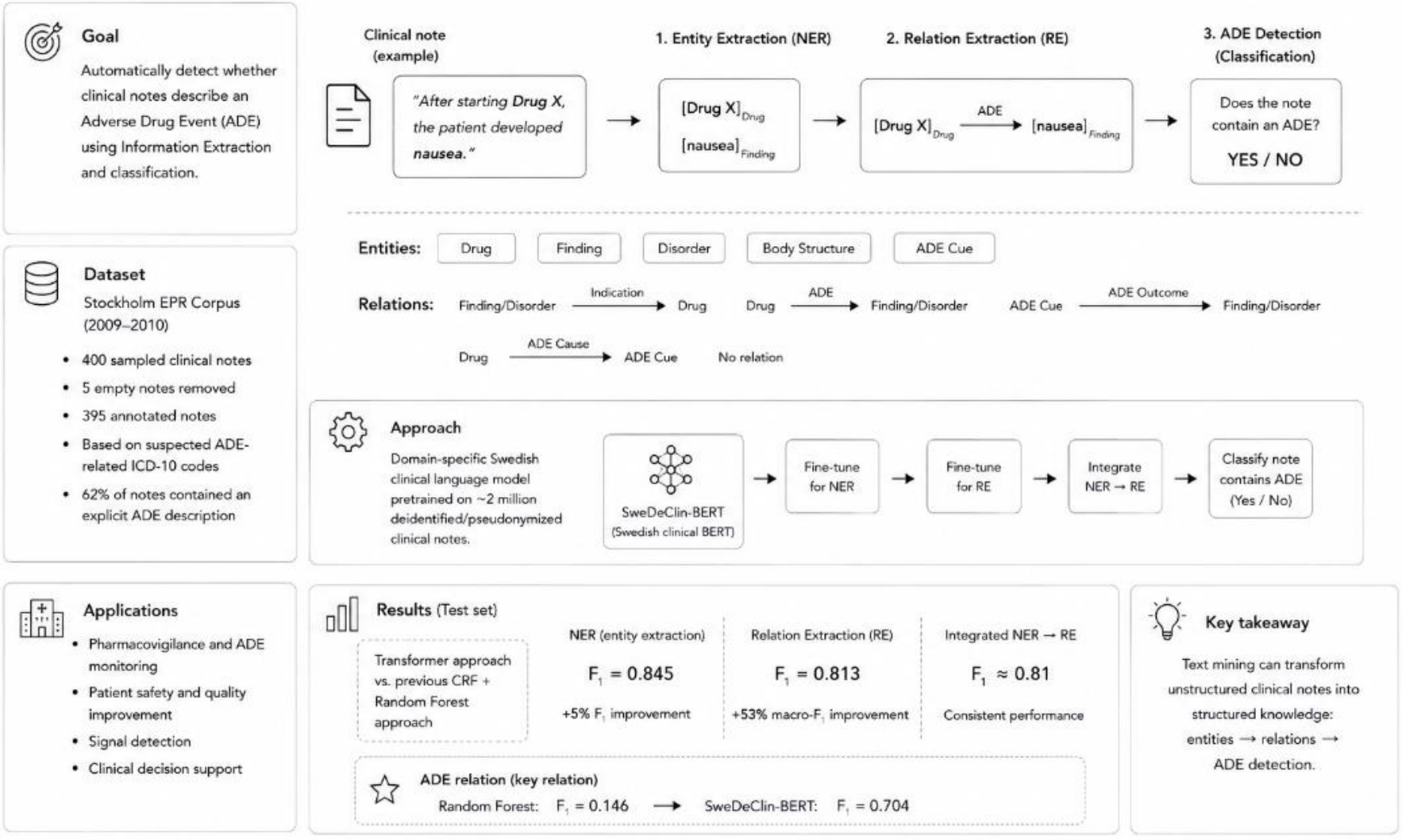
Gold: [severe abdominal pain]

Pred: [abdominal pain]

Use appropriate metrics:
Precision, Recall, F1, Partial match, Exact match depending on the task.

Finding an entity is not enough — we must understand its context and relationships.

Real Case: ADEs



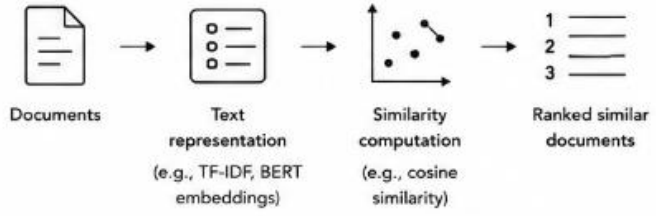
Kopacheva, E., Henriksson, A., Dalianis, H., Hammar, T. and Lincke, A., 2025. Fine-tuning Clinical Language Models to Identify Adverse Drug Events in Clinical Text: Machine Learning Approach (Preprint). JMI R Formative Research.

Find Relevant Information: Document Similarity

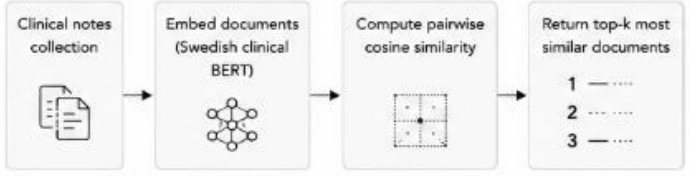
Goal

Find documents that are semantically similar to a given query document.

How it works



Approach (example)



Example

Query document

"Patient developed **nausea** and **vomiting** after starting **metformin**. Drug was stopped and symptoms resolved."

Top 3 most similar documents (by cosine similarity)

Rank	Similar document (snippet)	Similarity
1	"After initiation of **metformin** the patient experienced **nausea** and **vomiting**. Treatment discontinued with improvement."	0.86
2	"**Metformin** therapy was followed by **nausea**. The patient also had abdominal discomfort."	0.79
3	"Patient on **metformin** presented with **vomiting** and loss of appetite."	0.74

How similarity is computed

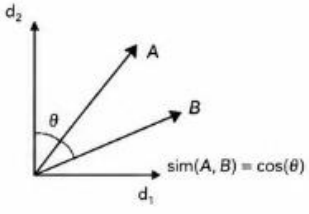
Common choices:

- Cosine similarity (most used)
- Euclidean distance

Cosine similarity

$$\text{sim}(A, B) = \frac{A \cdot B}{\|A\| \ \|B\|}$$

Range: 0 (not similar) to 1 (identical)



Applications (examples from literature)

- Information retrieval in electronic health records (SemEval 2016 ClinicalIR Task; Demner-Fushman et al.)
- Finding similar patient cases to support clinical decision making (Uzuner et al., JAMIA 2011)
- Pharmacovigilance: find reports similar to a known ADE case (van Puijenbroek et al., Drug Safety 2002)
- Cohort discovery and phenotype identification (Sun et al., JAMIA 2020)

Limitations & issues

- Depends on text representation and preprocessing
- May miss lexical variation or abbreviations
- Sensitive to domain shift and model choice
- Similarity ≠ causal or clinical relevance
- Short documents contain less information

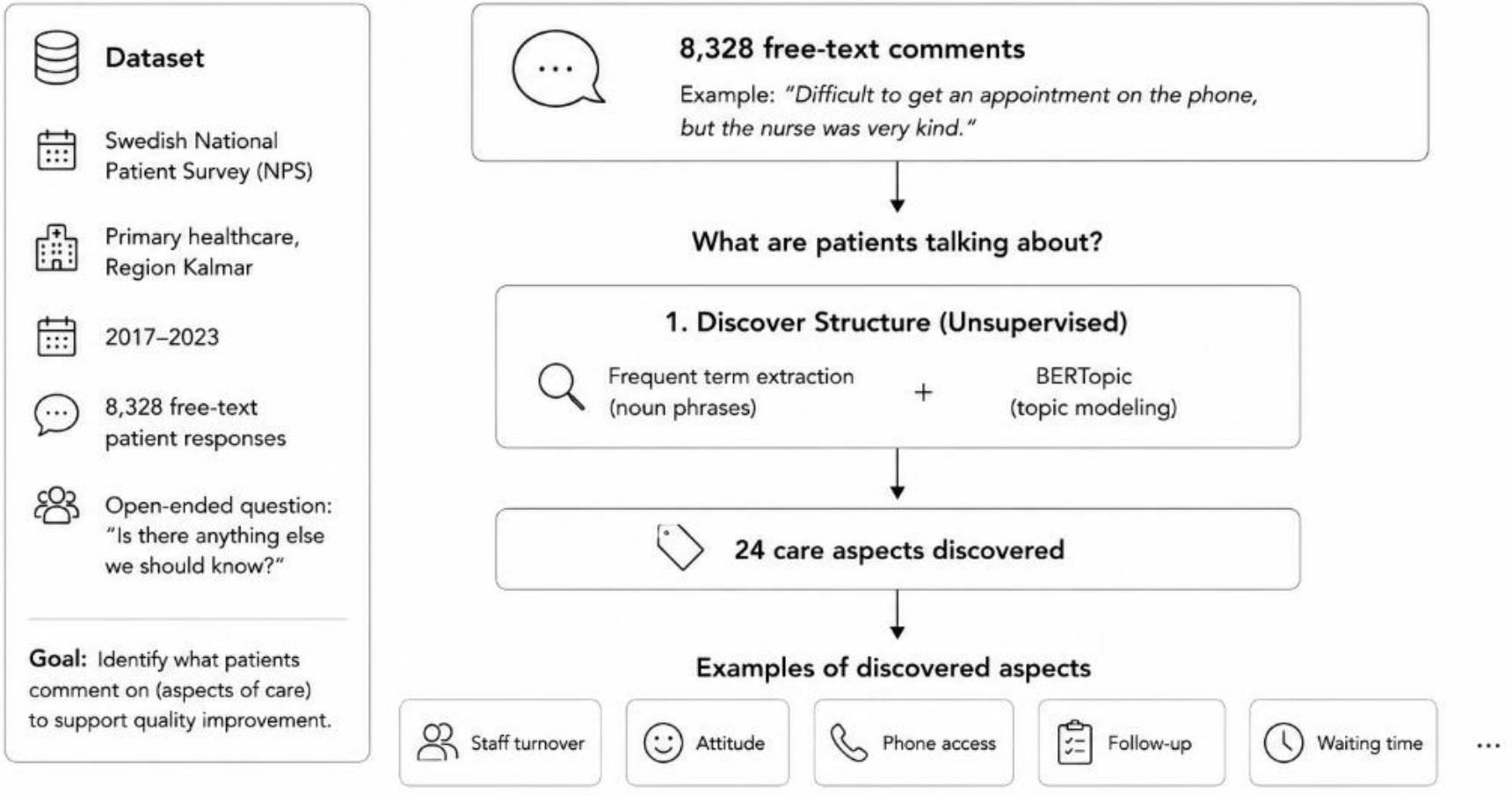
Evaluation

No single gold standard.

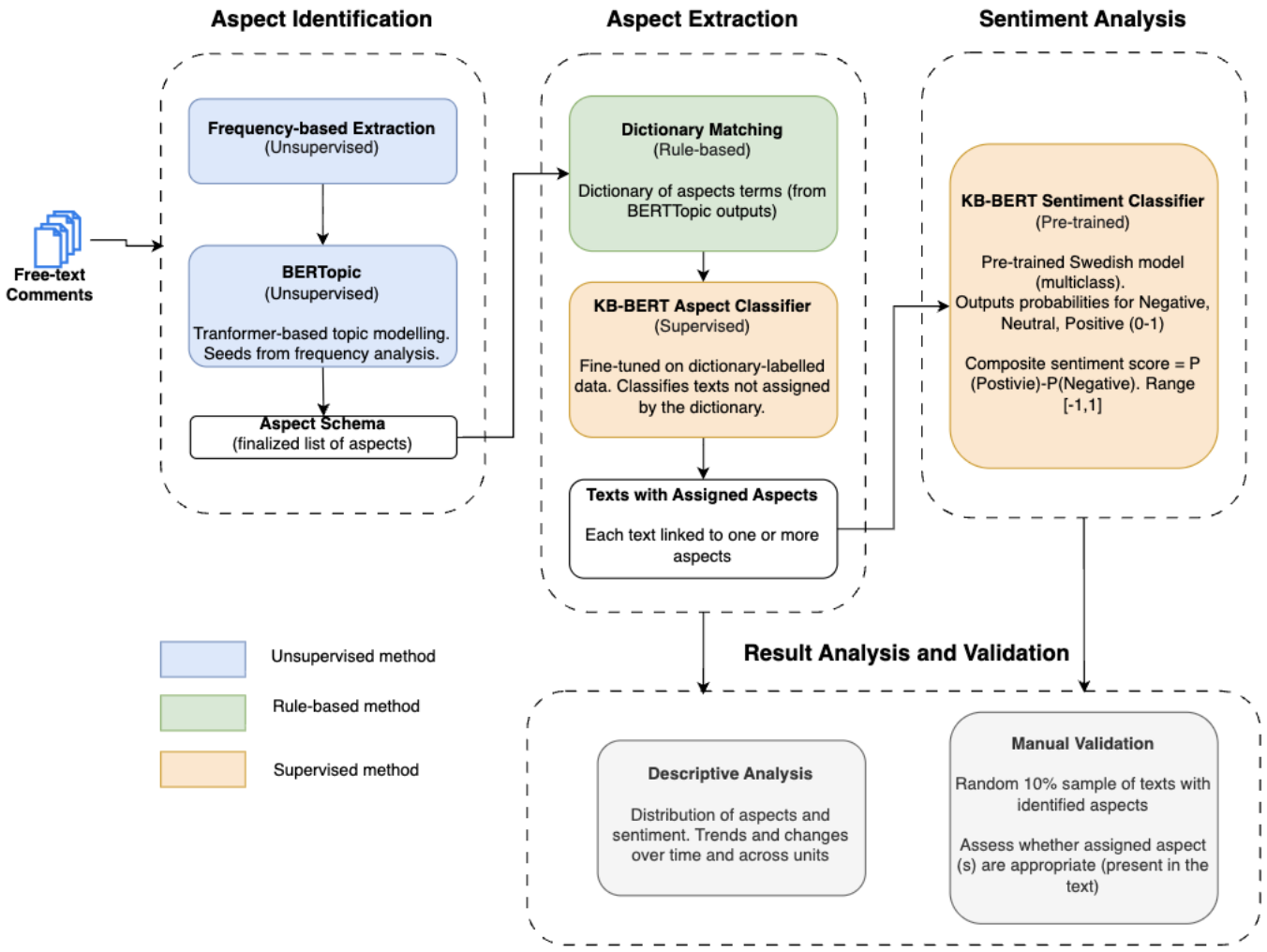
Common approaches:

- Human judgment / relevance assessment
- nDCG, MAP, Precision@k
- Intrinsic evaluation (cluster quality, etc.)

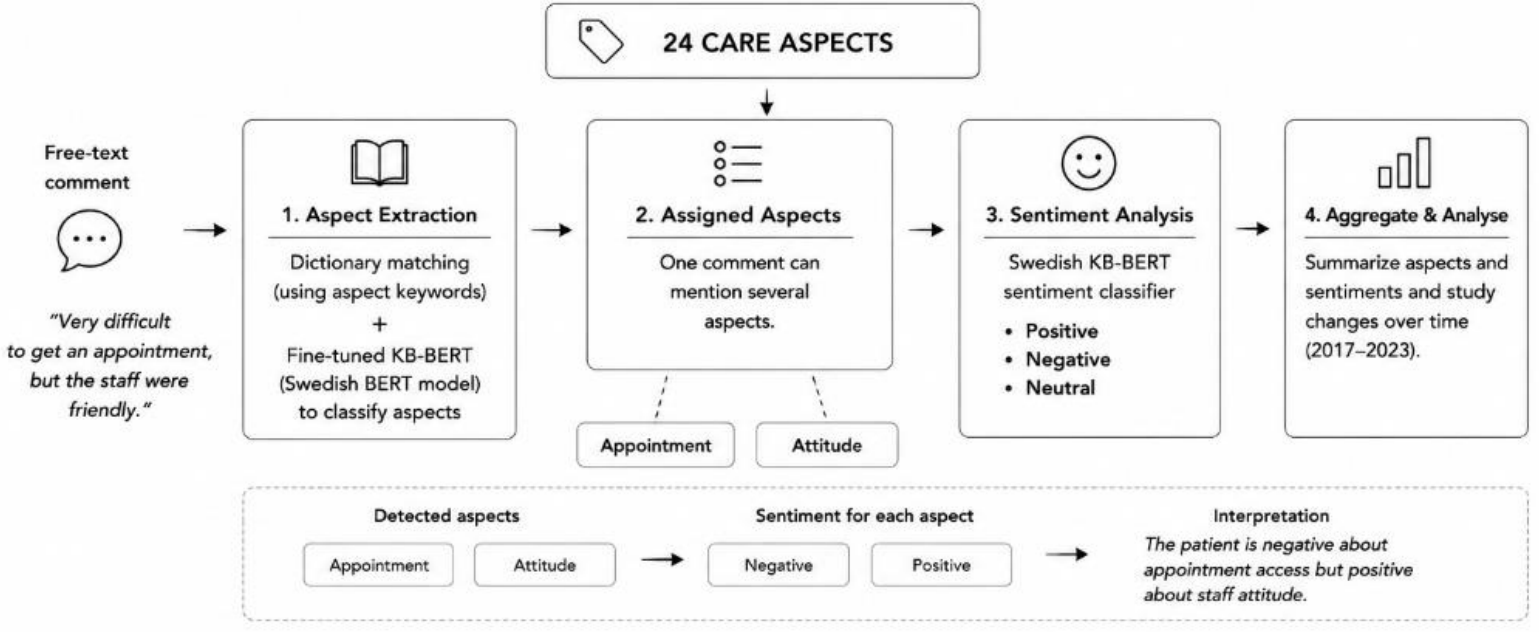
Real case: From patient comments to structured data



From patient comments to structured data

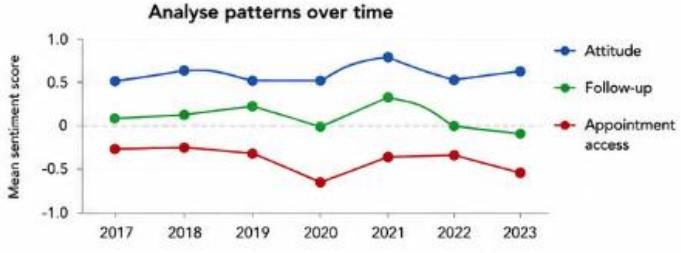


Real case: From patient comments to structured data



Structured data output
Each row = one aspect–sentiment assignment from a comment.

Aspect	Sentiment	Year	Comment ID
Appointment	Negative	2021	C000123
Attitude	Positive	2021	C000123
Follow-up	Neutral	2021	C000124
...	...	...	...



How do aspects and sentiments change over time?

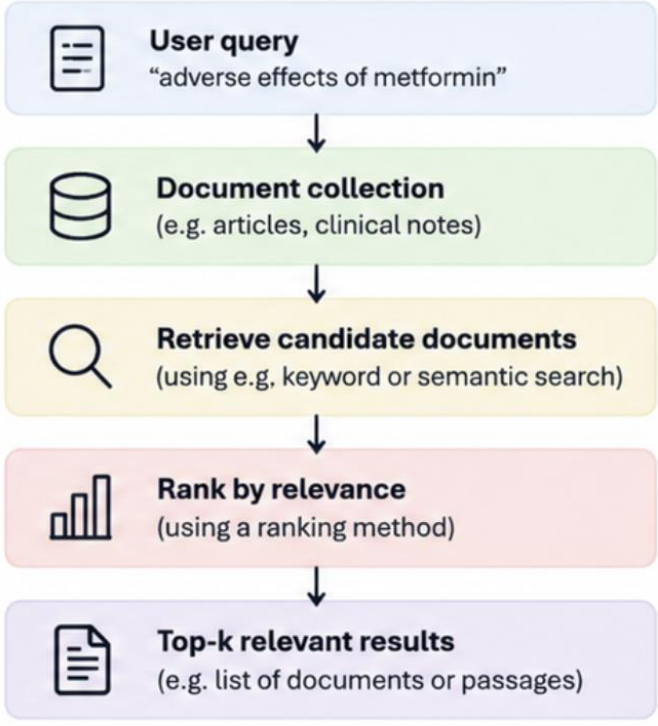
Find Relevant Information: Information Retrieval

- **Information retrieval (IR)** is the process of finding relevant documents or passages from a large collection in response to an information need (query).
- IR typically involves:
  - **representing** the query and documents
  - **retrieving** candidate documents
  - **ranking** them by relevance
- The goal is to return the **most useful information** to the user.

Examples of information retrieval

Web search	Search in scientific literature	Search in electronic health records	Enterprise document search
(e.g. Google)	(e.g. PubMed)	(e.g. clinical notes)	(e.g. company reports)

Typical IR workflow



Key point

Information retrieval helps us efficiently find the most relevant information in large collections of text.

Representing Queries and Documents

- To compare a query with documents, we first need to represent them in a form that a computer can process.
- Different representations capture different types of information (exact words vs. meaning).

Keyword (lexical) representation

e.g. Bag-of-Words, TF-IDF

- Represent text as a **sparse** vector of **term weights**.
- Focuses on **word overlap** between query and document.
- Used in traditional keyword search (e.g. BM25, TF-IDF + cosine similarity).

Query
"medication causing stomach problems"

Document
"Metformin caused gastrointestinal adverse effects."

Term	Query (TF-IDF)	Document (TF-IDF)
medication	0.42	0.00
stomach	0.38	0.00
problems	0.35	0.00
metformin	0.00	0.40
gastrointestinal	0.00	0.36
...	...	...

Similarity (lexical)

Often low, because few words overlap.

Semantic representation

e.g. word embeddings, sentence embeddings

- Represent text as a **dense** vector in a continuous space.
- Captures **semantic meaning** and context.
- Can find relevant documents even when the exact words differ.
- Used in modern semantic search and in RAG systems.

Query
"medication causing stomach problems"

Document
"Metformin caused gastrointestinal adverse effects."

Dimension	Query (embedding)	Document (embedding)
1	0.12	0.11
2	-0.34	-0.31
3	0.56	0.53
4	0.18	0.20
5	-0.27	-0.25
...	...	...

Similarity (semantic)

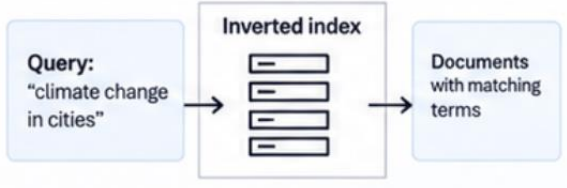
Can be high, because the meanings are related, even if the exact words are different.

Retrieve Candidate Documents

- After representing the query and document, we retrieve a set of candidate documents that may be relevant
- Different retrieval approach use different mechanisms to find candidates

1. Keyword-based retrieval
(e.g. Boolean search, inverted index)

- Match query terms with terms in the document index.
- Usually exact word matching (possibly with stemming, stop-word removal).
- Very fast and scalable.



Example retrieved candidates

D1: Climate change and urban development
D2: The impact of climate on agriculture
D3: Cities and sustainable transport
...

2. Traditional ranking methods
(e.g. TF-IDF, BM25)

- Compute a relevance score based on term frequency, term rarity and document length.
- Retrieve top-k documents with highest scores.
- Widely used in search engines and many applications.

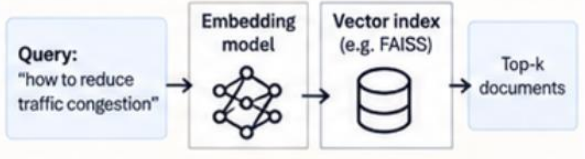


Example retrieved candidates

D1: Policy measures for renewable energy
D2: Energy markets and regulation
D3: Solar power production in Europe
...

3. Semantic retrieval
(dense vector search)

- Use pre-trained language models to create vector embeddings for query and documents.
- Find documents with similar meaning (even if different words are used).
- Retrieve top-k most similar documents using nearest neighbor search.



Example retrieved candidates

D1: Strategies for reducing urban traffic
D2: Urban mobility and sustainable cities
D3: Public transport planning guidelines
...

4. Hybrid retrieval / RAG
(lexical + semantic, used in RAG systems)

- Combine keyword-based and semantic retrieval (e.g. weighted combination or re-ranking).
- Often used as the first step in RAG (Retrieval-Augmented Generation).
- Aims to improve recall by leveraging both exact term matching and semantic similarity.

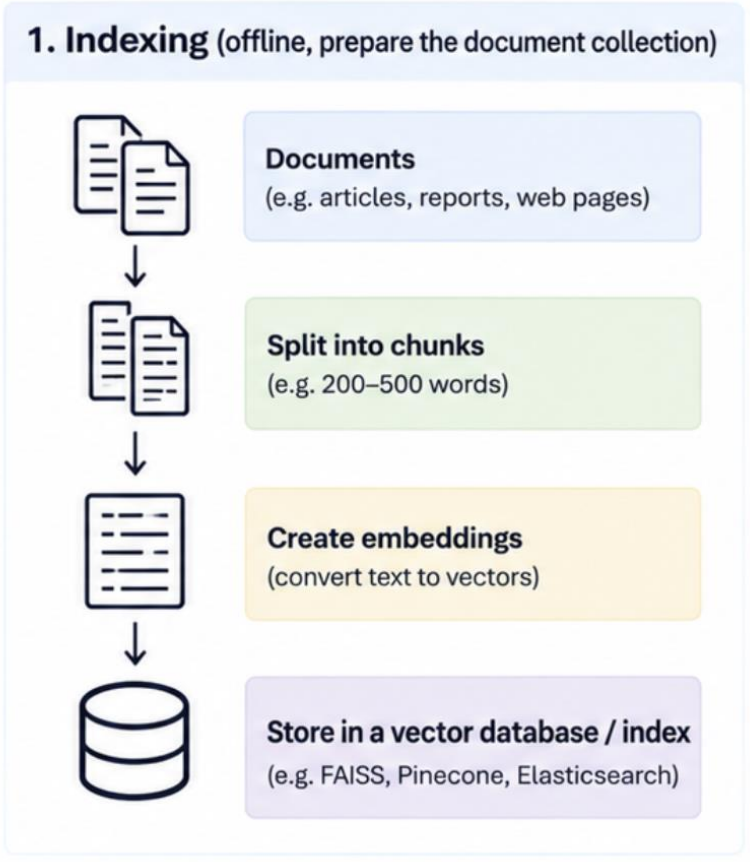


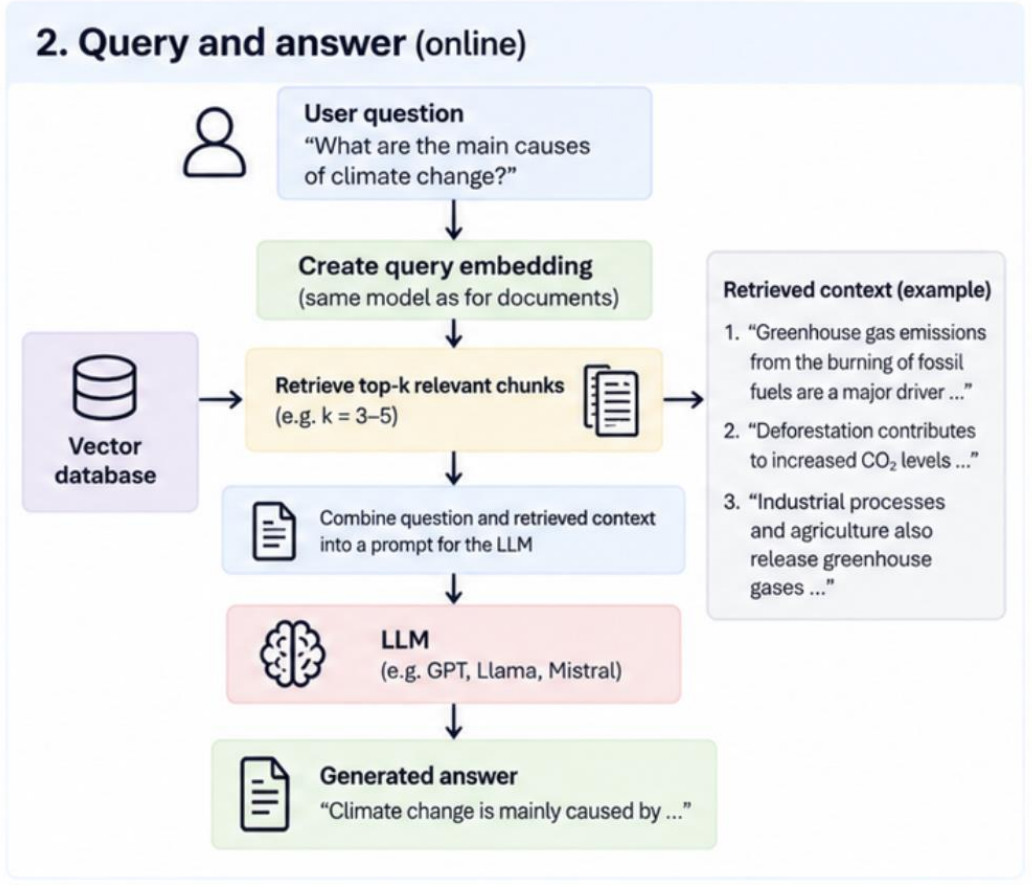
Example retrieved candidates

D1: Reducing plastic waste in coastal areas
D2: Innovative materials to replace plastic
D3: Policy measures for single-use plastics
...

How RAG works

- RAG combines an information retrieval system with a language model
- Relevant information is retrieved from an external document collection and provided as context to the LLM, which then generate an answer





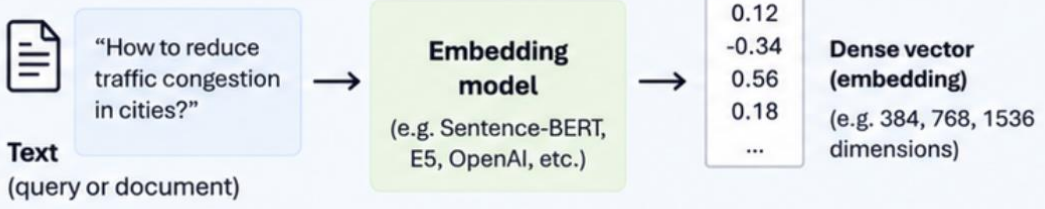
Key points

1. Documents are split into smaller chunks and converted to embeddings.
2. For a user question, a query embedding is created.
3. The system retrieves the top-k most relevant chunks from the vector database.
4. The retrieved chunks are added to the user question as context in a prompt.
5. The LLM generates an answer based on the retrieved information.
6. The quality of the answer depends on the quality of the retrieval.

Text Embeddings

- An embedding is a **dense numerical vector** that represents the meaning of a text (word, sentence, or document) in a continuous vector space.
- Texts with similar meaning are mapped to nearby vectors.
- The choice of embedding model depends on the task, data, and domain, and is **decided during the implementation**.

From text to vector



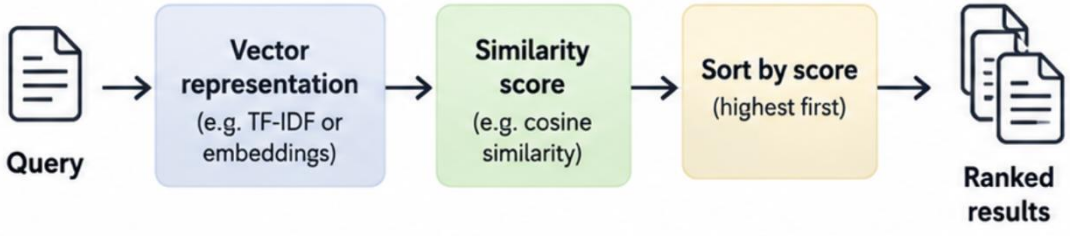
Common types of text embeddings

Type	Description	Examples	Typical use cases	Notes
Word embeddings	Vector for individual words (static embeddings).	Word2Vec, GloVe, fastText	Simple semantic similarity, classical NLP tasks.	One vector per word. Does not consider context (same vector for all word senses).
Contextual embeddings	Vector depends on the context in which a word appears.	BERT, RoBERTa, DistilBERT	General NLP tasks, can be adapted for sentence or document embeddings.	Produces contextual representations. Often need pooling to get a single sentence/document vector.
Sentence / document embeddings	Vector for a whole sentence or document.	Sentence-BERT (all-MiniLM, all-mpnet-base), E5, BGE	Semantic search, clustering, RAG.	Specifically trained to create high-quality sentence/document embeddings.
Domain-specific embeddings	Trained or fine-tuned on domain data (e.g. scientific, medical, legal).	BioBERT, ClinicalBERT, SciBERT, Legal-BERT	Search in specialized domains (e.g. scientific literature, clinical notes).	Better performance on domain-specific language and terms.
Multilingual embeddings	Support multiple languages in a shared vector space.	LaBSE, multilingual E5, BGE-m3	Cross-lingual search, multilingual RAG.	Useful when working with documents in different languages.
Commercial embeddings	Proprietary models provided as a service (API).	OpenAI text-embedding-3, Cohere Embed, Amazon Titan	High-quality embeddings, easy integration.	Good performance, but may involve cost and data privacy considerations.

Ranking

From Similarity Scores to Ranked Results

- **Ranking** = ordering documents by their estimated relevance to a query.
- We use a **similarity** (or **relevance**) **score** to measure how well each document matches the query.
- Documents are then **sorted** by their scores, with the most relevant documents first.



Example

Query: "adverse effects of metformin"

Rank	Document	Similarity score
1	D3	0.91
2	D1	0.78
3	D4	0.42
4	D2	0.16

Takeaway

Ranking provides an ordered list of documents, so that the most relevant information appears first.

Evaluation in Information Retrieval

Classical Information Retrieval

Evaluate the retrieved (and ranked) documents

Relevance judgement

- Compare retrieved documents with ground truth relevant documents.
- Use metrics that consider precision, recall and ranking quality.

Example: search results

Query: "renewable energy policy"

1. ✓ Relevant
2. ✓ Relevant
3. ✗ Not relevant
4. ✓ Relevant
5. ...

Common metrics

Metric	Formula / Idea	Interpretation
Precision@k	$\frac{\#\text{ relevant in top-k}}{k}$	Fraction of retrieved documents that are relevant.
Recall@k	$\frac{\#\text{ relevant in top-k}}{\#\text{ relevant in total}}$	Fraction of all relevant documents that are retrieved.
F1@k	$\frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$	Harmonic mean of precision and recall.
MAP (Mean Average Precision)	Average precision over all queries	Considers ranking of relevant documents.
nDCG (normalized DCG)	Rewards relevant documents higher in the ranking (graded relevance).	Measures ranking quality, with higher scores for relevant documents at top.

Evaluation in RAG Systems

Evaluate both retrieval and generated answer



1. Retrieval evaluation (same as classical IR)

Evaluate the quality of the retrieved context.

Metric	What it measures
Precision@k	How many of the retrieved documents are relevant.
Recall@k	How many of the relevant documents are retrieved.
MAP / nDCG	How well relevant documents are ranked.

2. Generation evaluation

Evaluate the quality of the generated answer (using the retrieved context).

Metric / Method	What it measures
Faithfulness	Is the answer supported by the retrieved documents? (e.g. factual consistency)
Answer relevance	Does the answer address the user's question?
Groundedness	Extent to which the answer is based on the retrieved context.
Helpful/fluency	Is the answer clear, complete and useful? (often human or LLM-as-a-judge evaluation)

Example (RAG)

Query: "How does urban green space affect mental health?"
Retrieved documents: 3 relevant, 2 not relevant → Precision@5 = 0.6, Recall@5 = 0.75
Generated answer: Factually correct, answers the question, and is supported by the retrieved sources
→ High faithfulness → High answer relevance

Summary

- **Text mining** transforms unstructured text into information that can be analyzed and used.
- Text is typically **preprocessed** and converted into a numerical **representation**.
- Common representations include **Bag of Words, TF-IDF, n-grams, word embeddings, and contextual embeddings**
- Depending on the objective, text mining can be used to **predict/classify, discover structure, extract information, or find relevant information**.
- Evaluation depends on the task: **classification/extraction metrics, clustering evaluation, retrieval metrics, and RAG answer quality**.